LUMO SAMJHO

Complete ZIP File Description

A file-by-file guide to the application structure, e-Shram RAG workflow, Sarvam AI integration, BHASHINI features, data files, scripts, tests, and setup.

Purpose

This guide explains the role of each major file and folder in the uploaded LUMO Samjho project and shows how the parts fit together.

Important distinction

The project uses Retrieval-Augmented Generation (RAG) to provide Sarvam with relevant document passages at question time. RAG does not change the model's weights and is not the same as fine-tuning. The knowledge base must contain verified official source content before answers can be grounded in it.

1. Project Structure

The project is organised into the application code, source documents, scripts, tests, configuration, and dependency files.

Path	Role
app/	Core FastAPI application, RAG, Sarvam/BHASHINI integration, security, schemas, and data.
app/data/	Local glossary, scheme information, and searchable document chunks.
documents/	Source-document configuration and instructions for official e-Shram material.
scripts/	Fetch, ingest, evaluate, check provider connectivity, and export candidate training data.
tests/	Automated tests and sample retrieval-evaluation questions.
README.md	Setup, architecture, API usage, and development instructions.
requirements.txt	Runtime dependencies.
requirements-dev.txt	Development and test dependencies.
.env.example	Example configuration; copy to .env and add secrets privately.
.gitignore	Excludes secrets, caches, environments, and selected generated files from Git.
.vscode/	Editor-specific settings.
__MACOSX/	macOS ZIP metadata; not application code.

2. Application Files: app/

File	Purpose
main.py	Main FastAPI application. Registers routes, validates incoming requests, connects the modules, and returns API responses.
config.py	Reads environment settings such as API keys, model identifiers, timeouts, retrieval thresholds, CORS, and security options.
rag.py	Coordinates retrieval-augmented generation: finds relevant document passages, builds model context, requests an answer, and returns source metadata or a fallback.
knowledge.py	Manages knowledge-base chunks and retrieval ranking, plus knowledge-base status/summary helpers.
sarvam.py	Wraps calls to Sarvam's API, including request headers, response parsing, retry and error handling.
llm.py	Language-model helpers for explaining or simplifying financial terms.
bhashini.py	Optional BHASHINI integration for supported translation and text-to-speech operations.
schemas.py	Pydantic request and response schemas used to validate API payloads.
security.py	API-key protection, request access controls, and rate limiting.
pii.py	Detects or redacts personally identifiable information to reduce privacy risks in logs or model requests.
textutil.py	Text cleaning, tokenisation, spelling normalisation, and language-related retrieval helpers.
glossary.py	Loads and searches the financial glossary.
resources.py	Provides resource cards and related information through API endpoints.
__init__.py	Marks app as a Python package and may define package-level exports.

3. Data Files: app/data/

File	Purpose	Operational note
glossary.json	Structured financial terminology, definitions, simpler explanations, categories, and aliases.	The app can use this without training a model.
schemes.json	Structured scheme-related information and eligibility references. Verify all details against current official sources.	Treat this as reference data, not a guarantee of eligibility.
knowledge_base.json	Searchable document chunks used by RAG, with text and source metadata such as titles, URLs, dates, pages, or chunk IDs when available.	The uploaded archive's knowledge base starts empty ([]) and must be populated by ingesting source documents.

RAG vs fine-tuning

RAG retrieves source passages and supplies them to the model for each question. Fine-tuning uses examples to update a model through a supported training process. Updating JSON documents does not fine-tune Sarvam or change its model weights.

4. Scripts/

Script	Purpose
bootstrap.py	Runs the bundled initialisation workflow for source loading, retrieval evaluation, and Sarvam checks.
fetch_sources.py	Downloads source material defined in documents/sources.json; review failed downloads and empty responses.
ingest_docs.py	Extracts document text, splits it into overlapping chunks, attaches metadata, and writes to the knowledge base.
eval_retrieval.py	Checks whether sample questions retrieve relevant passages.
check_sarvam.py	Checks provider configuration and tests model requests, subject to valid credentials and supported endpoint settings.
export_finetune_data.py	Prepares candidate question-answer examples for a possible future fine-tuning experiment; output must be reviewed before training.

5. Documents and Source Configuration

File	Purpose
documents/README.md	Instructions for adding official documents and source metadata.
documents/sources.json	Configured official sources to fetch and ingest.
documents/sources.example.json	Example source configuration for customisation.

A URL in a configuration file does not mean the content has been downloaded or indexed. Fetch the source, inspect extraction quality, and ingest it before expecting RAG to use it. Scanned PDFs may require OCR.

6. Tests/

File	Purpose
conftest.py	Shared fixtures, temporary knowledge bases, and mocked provider settings.
test_api.py	API routes, payload validation, glossary behaviour, API-key enforcement, rate limits, and status checks.
test_retrieval.py	English, Hindi, Hinglish, spelling variants, multilingual retrieval, and out-of-scope questions.
test_ingest.py	Document extraction, PDF page metadata, HTML, repeat ingestion, source fetching, and export behaviour.
test_rag.py	Answer generation, empty knowledge base, fallbacks, citations, multilingual requests, privacy handling, and API errors.
eval_questions.json	Sample questions and expected keywords for retrieval evaluation.

Run the tests with:

```
pip install -r requirements-dev.txt
pytest -q
```

The README reports 47 tests. Passing tests verifies software behaviour, not the factual correctness of every government-policy answer. Verify answers against current official sources.

7. Configuration and Dependencies

File	Purpose
.env.example	Template for API keys, Sarvam endpoint/model settings, timeouts, retrieval thresholds, optional API-key protection, rate limits, CORS, and BHASHINI settings.
requirements.txt	Runtime packages such as FastAPI, Uvicorn, HTTPX, Pydantic, python-dotenv, and pypdf.
requirements-dev.txt	Runtime dependencies plus testing and development packages, including pytest and ReportLab.
.gitignore	Keeps secrets, environments, caches, and selected generated files out of Git.
.vscode/settings.json	Editor settings only; does not change AI logic.

Security note

Keep API credentials in .env, never commit them, avoid ingesting private worker records, and protect any personal data. Confirm the Sarvam endpoint and model identifier against the current account documentation. BHASHINI features remain optional when access is pending.

8. API Endpoint Summary

Endpoint	Purpose
GET /health	Basic application health check.
GET /api/v1/glossary	Returns glossary entries.
POST /api/v1/explain	Explains a financial term.
POST /api/v1/ask	Answers e-Shram questions through RAG.
GET /api/v1/knowledge/status	Reports loaded knowledge-base status.
POST /api/v1/llm/check	Checks language-model integration.
POST /api/v1/translate	Optional BHASHINI translation.
POST /api/v1/speech	Optional BHASHINI speech generation.
GET /api/v1/resources	Returns resources and supported filters.

9. Setup and Run Instructions

From the extracted project directory on macOS:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
cp .env.example .env
```

Edit .env and set valid provider credentials and supported model settings. Then fetch and ingest official source content:

```
python scripts/fetch_sources.py
python scripts/ingest_docs.py documents/
python scripts/eval_retrieval.py
python scripts/check_sarvam.py
pytest -q
uvicorn app.main:app --reload
```

Open the interactive API documentation at http://127.0.0.1:8000/docs after the server starts.

10. Recommended Development Priorities

Priority	Action
1. Verify sources	Check documents/sources.json and ensure each source is official, current, and reachable.
2. Populate knowledge	Fetch, inspect, and ingest the documents; confirm knowledge_base.json contains meaningful chunks.
3. Evaluate retrieval	Run the retrieval evaluator and add real Hindi, Hinglish, and English questions.
4. Verify model calls	Check Sarvam endpoint, model ID, request format, and API key against current provider documentation.
5. Validate factual accuracy	Review answers against official pages and include clear source details and dates.
6. Consider fine-tuning later	Only export reviewed, high-quality examples if there is a specific training objective and a supported fine-tuning route.

Final takeaway

The project is structured to combine official e-Shram information with Sarvam-generated responses. The most important next task is to load and verify the official knowledge base. RAG improves source grounding without changing the model weights; fine-tuning is a separate optional future step.

This guide describes the project structure and workflow. Actual file contents, provider API compatibility, downloaded source availability, and live API behaviour should be verified in the extracted project and current provider documentation.